

TriGuard: Validity Aware Caching for Gen AI

Aman Soni
CSE-584, University of Michigan
Ann Arbor, USA
amasoni@umich.edu

Meghanand Gejjela
CSE-584, University of Michigan
Ann Arbor, USA
meghss@umich.edu

Prudvi Sai Padidala
CSE-584, University of Michigan
Ann Arbor, USA
prudvi@umich.edu

Abstract

As the adoption of Generative AI via API gateways accelerates, the latency and financial cost of processing redundant, naturally phrased queries has become a massive infrastructural bottleneck. Traditional exact-match caching is completely insufficient for natural language, whereas existing semantic caching solutions rely almost exclusively on simple cosine similarity with rigid thresholds. This leads to brittle systems that suffer from high false-positive rates and lack temporal awareness. To address these critical shortcomings, we introduce TriGuard, an advanced tri-gated semantic cache pipeline architecture. TriGuard sequences incoming requests through three primary gates: Context Normalization, Temporal (TTL) Classification, and Triple Small Language Model (SLM) Verification. Furthermore, TriGuard operates fundamentally on a dual storage architecture leveraging both Redis and Chroma DB. A central novelty of our approach is the systematic replacement of deterministic cosine similarity with Pythagorean Fuzzy Sets (PyFS), acting as a mathematically robust alternative for assessing semantic closeness under uncertainty. This paper details our problem statement, explores the mathematical foundations of PyFS, details the system architecture, and sets the stage for extensive experimental evaluation of cache hit rates and false-positive suppression.

Keywords

Semantic Caching, Large Language Models, Pythagorean Fuzzy Sets, Temporal Classification, Context Normalization, Triple SLM Verification, Chroma DB

1 Introduction

The integration of Large Language Models (LLMs) into production environments typically relies on third-party APIs such as OpenAI’s GPT-4 or Google’s Gemini. For large-scale production systems, the repeated execution of semantically identical queries—phrased differently by different users—incurs severe computational redundancy. This redundancy scales linearly with user traffic, resulting in exorbitant financial costs and unacceptable user-facing latency. Conventional web caching strategies rely on exact string matching, which fail completely in the context of natural language where identical intents can be articulated in a nearly infinite number of structural ways.

Semantic caching was introduced to solve this exact problem. By projecting queries into a multidimensional vector space using dense retrieval models, systems can evaluate intent equivalence. However, current implementations, such as the widely adopted GPTCache framework, operate on a highly linear and simplistic assumption: a query is embedded, compared against prior queries, and accepted if the *cosine similarity* exceeds a predetermined geometric threshold.

This mechanism assumes that geometric embedding distance strictly correlates with true semantic equivalence. In practice, this

assumption is notoriously flawed. Subtle negations or nuanced contradictions often trick the system into serving completely incorrect cache hits. Furthermore, traditional semantic caches are "temporally blind." A query like "What is the stock price of Apple?" requires a vastly different Time-To-Live (TTL) cache duration than "What year did Apple launch the iPhone?"

In this paper, we present **TriGuard**, a significantly advanced semantic cache tailored for complex Generative AI workloads. TriGuard abandons simplistic single-layer checks in favor of three robust operational gates: **Context Normalization**, **Temporal Classification (TTL)**, and **Triple SLM Verification**.

Critically, we introduce the application of **Pythagorean Fuzzy Sets (PyFS)** not as a separate gate, but as the foundational mathematical replacement for cosine similarity. PyFS allows us to precisely quantify semantic closeness while explicitly modeling system uncertainty and hesitancy (π), leading to drastically lowered false positive rates. Coupled with a dual-storage search architecture spanning Redis and Chroma DB, TriGuard provides a highly scalable and mathematically safe caching protocol.

2 Background and Motivation

2.1 Challenges in Exact Match Caching

Traditional web infrastructures rely on caching architectures like Redis, which hash the payload exactly. In GenAI, the input is natural language. Consider the queries "How much ibuprofen should I take?" and "What is the recommended dosage for ibuprofen?" These strings share very few characters yet demand the exact same response. Exact string matching results in cache miss rates approaching 100% for conversational AI platforms.

2.2 The Fragility of Cosine Similarity

To counter natural language variability, first-generation semantic caches embed the incoming string into a float vector and perform an approximate nearest neighbor search. The motivation for TriGuard stems from observing the fragility of these dense retrieval layers. When systems rely on fixed cosine similarity thresholds, administrators face a strict dichotomy. They can set the threshold aggressively high to ensure safety, thereby minimizing false positives but rejecting synonymous paraphrases. Alternatively, they can lower the threshold to boost the hit rate, but risk serving completely incorrect data when vectors cluster tightly around a topic despite opposing semantic goals. A robust mathematical alternative to cosine similarity for modeling textual relationships is desperately needed.

3 Related Work

The landscape of semantic caching has seen rapid development as organizations attempt to alleviate LLM deployment costs. We categorize prior work into three generations of architectures.

3.1 First-Generation Semantic Caches

GPTCache (2023) and **MeanCache (2024)** are the foundational frameworks for modern semantic caching. They utilize pre-trained dense embeddings (often OpenAI’s text-embedding-ada-002 or open-source equivalents) and a basic nearest-neighbor vector store like FAISS or Milvus. They trigger cache hits based entirely on single-stage cosine similarity against a static threshold. Their massive limitation is the lack of secondary verification. Simple negations (“I do not like apples” vs “I like apples”) often embed too close together, tricking the cache.

3.2 Dynamic Thresholding and Eviction

To address the inflexibility of fixed thresholds, recent systems have explored dynamic boundaries. **SISO (2025)** introduced bandwidth-aware semantic eviction. It adjusts thresholds dynamically based on cache load, lowering the threshold during high API traffic to shed load, and raising it during low traffic to ensure accuracy. **Liu et al. (2025)** proposed a cost-aware framework that incorporates mismatch costs (the penalty of a false positive) and learns optimum thresholds via multi-armed bandit techniques. **LangCache (2025)** attempts to improve raw embedding quality through domain-specific contrastive fine-tuning and integrates Time-To-Live (TTL) capabilities.

3.3 Multi-Stage and Asynchronous Verification

The closest contemporary research to our work includes **Krites (Feb 2026)** and **SphereLFU (Feb 2026)**. SphereLFU relies on a novel Kernel Density Estimation (KDE) prototype eviction mechanic to manage the vector space dynamically. Krites et al. propose offloading borderline cosine scores to an asynchronous LLM judge, acting as a secondary checkpoint.

However, none of these architectures employ a rigorous logical framework for handling inherent semantic ambiguity mathematically. TriGuard bridges this structural gap by integrating Pythagorean Fuzzy Logic to deterministically map model uncertainty across a multi-stage cascade.

4 Mathematical Foundations: PyFS instead of Cosine Similarity

The core theoretical novelty driving TriGuard’s retrieval accuracy is the total displacement of standard cosine similarity metrics with Pythagorean Fuzzy Sets (PyFS). PyFS acts as our primary measure of semantic closeness.

4.1 The Limitation of Geometric Distance

Cosine similarity only measures the angle between two multi-dimensional vectors. It does not measure logical contradiction. In Natural Language Inference (NLI), an element A and element B can possess a high degree of intersectional vocabulary but maintain mutually exclusive truths.

4.2 Pythagorean Fuzzy Sets for Closeness

Where classical fuzzy systems use a single degree of membership (μ) to denote closeness, Intuitionistic Fuzzy Sets (IFS) rely on the constraint $\mu_A(x) + \nu_A(x) \leq 1$, explicitly modeling non-membership (ν).

Pythagorean Fuzzy Sets (PyFS) mathematically expand the operable domain by relaxing the constraint to a quadratic boundary:

$$\mu_A(x)^2 + \nu_A(x)^2 \leq 1 \quad (1)$$

The hesitancy or uncertainty index $\pi_A(x)$, representing the system’s explicit lack of knowledge regarding the similarity, is defined via the Pythagorean theorem:

$$\pi_A(x) = \sqrt{1 - \mu_A(x)^2 - \nu_A(x)^2} \quad (2)$$

4.3 Operationalizing PyFS

Within TriGuard, when candidate vectors are retrieved from the database, we do not rank them mechanically by $L2$ norm or Cosine distance. Instead, we map the regression output logit (L) from an STS (Semantic Textual Similarity) model directly into PyFS parameters:

$$\mu = \sigma(L) \quad (3)$$

$$\nu = 1.0 - \sigma(L) \quad (4)$$

$$\pi = \sqrt{1 - \mu^2 - \nu^2} \quad (5)$$

Here, μ represents the explicit degree of semantic equivalence, ν represents the degree of contradiction, and π quantifies uncertainty. TriGuard uses these multi-dimensional scores to rank closeness far more safely than scalar cosine values.

5 The TriGuard Architecture

The TriGuard architecture is constructed around a highly optimized dual-storage paradigm integrated with three deeply specialized sequential gates. Our system separates fast exact-match lookups from complex semantic evaluations. Figure 1 demonstrates the rigorous data flow pipeline from the user’s prompt through cache validation.

5.1 Gate 1: Context Normalization

Natural language interactions within Generative AI form continuous dialogue sessions, making individual queries contextually dependent. Isolated queries arriving at the cache layer (e.g., “When was he born?”) are inherently ambiguous and dilute embedding quality within the vector store.

TriGuard intercepts incoming traffic via **Context Normalization** (Gate 1). This computationally standardizes the query utilizing a rolling window of the user’s conversational history (specifically, the last three dialogue turns). The module actively resolves dangling coreferences by injecting explicit entity states into the new query. Thus, “When was he born?” is rewritten transparently to “When was the CEO of Apple born?”. Furthermore, we track a specific metric evaluating exactly how often this rewrite engine triggers, isolating context-free prompts from highly dependent prompts. This guarantees that queries can be structurally embedded and matched independent of their relative conversational chain.



Figure 1: The TriGuard Architectural Flow. Queries pass through Context Normalization, are assigned cache lifetimes via Temporal TTL Classification, undergo semantic evaluation in Dual Storage via PyFS, and new answers are ultimately audited by an asynchronous Triple SLM Verification layer.

5.2 Gate 2: Temporal Classification (TTL)

The second boundary prevents the caching of fundamentally transient data. The **Temporal Classification** gate acts as the Time-To-Live (TTL) governor. Instead of uniformly caching every LLM generation, TriGuard utilizes a serialized Machine Learning classifier (`t1l_classifier.joblib`) trained specifically to infer the temporal half-life of a semantic intent.

- **Static/Slow-Moving Data:** Queries such as "What is the capital of Japan?" are assigned a "Static" class (infinite lifespan, $TTL = -1$) or a "Slow-Moving" class (e.g., 3 to 30 days expiry).

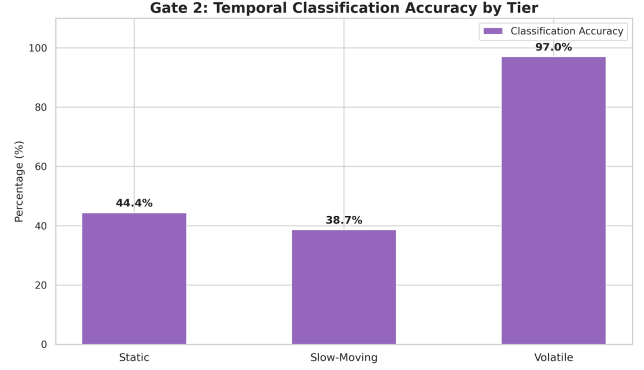


Figure 2: Custom Model Selectivity

Table 1: Performance of the Adaptive TTL Classifier across data volatility categories.

Category	TP	TN	FP	FN	Recall	F1
Volatile	32	60	7	1	0.970	0.889
Static	16	63	1	20	0.444	0.604
Slow-Moving	12	68	1	19	0.387	0.545

- **Volatile Data:** Queries containing implicit transiency (e.g., "What is the current stock price of TSLA?" or "What is the weather right now?") are explicitly flagged as "Volatile".

Crucially, if Gate 2 identifies a "Volatile" query, TriGuard *completely bypasses the semantic cache* (Chroma DB) and routes the request directly to the upstream GenAI model (e.g., Gemini). Serving a stale semantic cache hit for a live sports score or stock ticker is a catastrophic failure; Gate 2 structurally guarantees safety against hallucinatory chronological caching by mapping expiry constraints to the intent itself.

5.3 Dual Storage Search (Redis & Chroma DB)

For queries approved for caching, TriGuard executes a highly parallelized dual-database lookup utilizing asynchronous `get_event_loop()` executions.

Redis (The Fast Path Level-1 Cache): Redis functions as an ultra-low latency physical hash match. TriGuard applies an MD5 cryptographic hash to the normalized string. If a perfect collision occurs, the system completely bridges the NLP processing and serves the cached response within 15 milliseconds, respecting the TTL restrictions naturally enforced by Redis's `setex` mechanics.

Chroma DB & Embedding LRU (The Semantic Vector Store): Simultaneously, the query is embedded via the `bge-small-en-v1.5` model. To drastically reduce latency overhead, TriGuard maintains an application-layer Least Recently Used (LRU) embedding cache holding the last 1,024 vector tensors. If the string has been embedded recently, pulling the cached tensor saves approximately 120 milliseconds of forward-pass inference.

The embedded vector is dispatched to a persistent Chroma DB instance utilizing approximate nearest neighbor search via normalized cosine space (`hnsw:space: cosine`). However, the retrieved

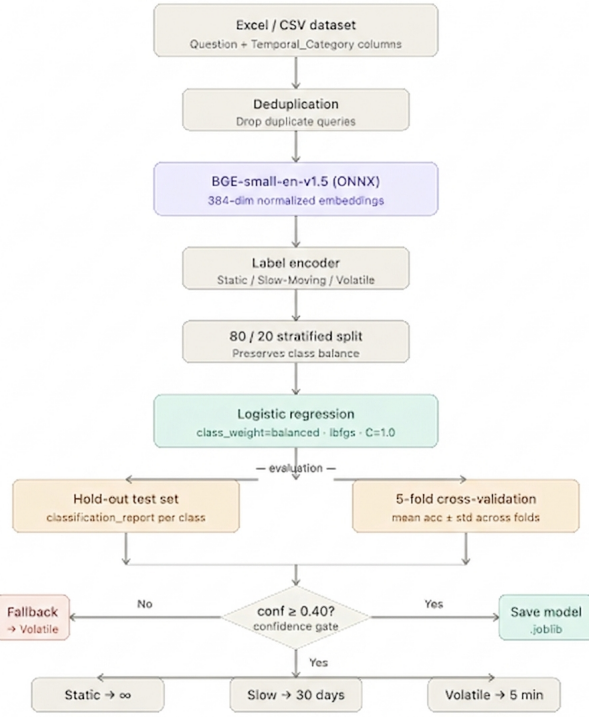


Figure 3: Training pipeline for training the embeddings of the custom TTL model which uses Logistic Regression over the text embeddings of the tokenizer for results with a good balance of accuracy and latency

candidates are simply considered mathematical 'potentials'. TriGuard immediately pipelines these candidates through our mathematical PyFS conversion (Section 4). High similarity vectors (Cosine > 0.80) are accepted via a "Fast Path", while closer, overlapping vectors are reranked via PyFS parameters enforcing strict non-membership/contradiction tolerances ($v \leq 0.30$).

5.4 Gate 3: Asynchronous SLM Verification

The final stop for protecting the integrity of the cache is **Gate 3: Triple SLM Verification**. Unlike Gates 1 and 2 which operate before cache retrieval, Gate 3 operates as an asynchronous backward-facing firewall *after* the primary GenAI LLM (Gemini) generates a fresh response for a cache miss.

Before injecting this freshly generated response into Redis and Chroma DB, we subject it to parallel Small Language Model (SLM) audits targeting exactitude.

- (1) **Intent & Relevance Verifier:** A retrieval-centric ranker (ms-marco-MiniLM-L-6-v2) validates standard passage relevance.
- (2) **Confidence & Faithfulness SLM:** A local dense model (Qwen2.5-1.5B) executed precisely to determine if the generated text is internally faithful and correctly answers the constraints of the query without meandering.

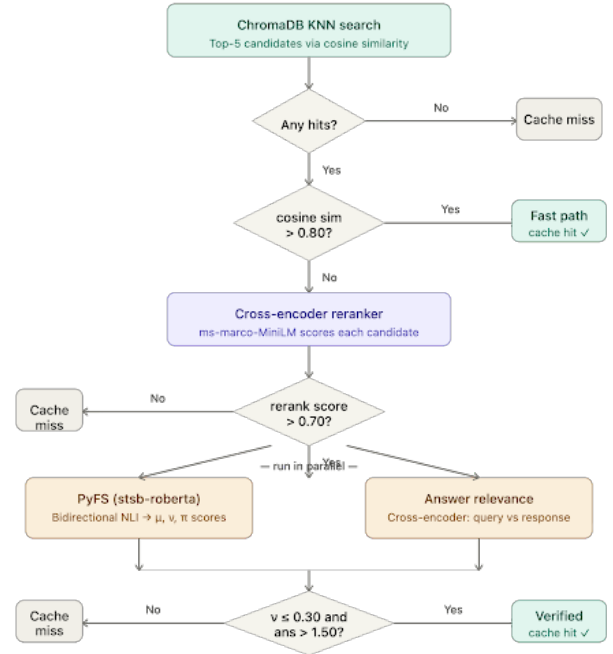


Figure 4: Semantic Similarity comparison pipeline which include an additional PyFS to ensure more robustness over traditional Cosine Similarity

Only if both background judgments yield a positive consensus does the response pass into the Chroma DB for future users. If Gate 3 observes hallucination or low-faithfulness, the response is served to the current user but actively *blocked* from being permanently cached, algorithmically protecting the storage layer from garbage data pollution.

6 System Implementation Rules

The realization of the Tri-Guard architecture adheres to several strict software engineering implementations:

- (1) **Concurrency and Write-Locks:** As Chroma DB and Redis are actively updated via background tasks (Gate 3), the Dual Storage Layer enforces thread-safe write locks during database insertion to prevent collision corruption under high parallel API throughput.
- (2) **Fail-Safe Logic:** Within Gate 2, if the Temporal Classifier fails to load or execution throws an unhandled exception, TriGuard mechanically defaults the query to "Volatile," failing gracefully by forcing a safe cache-miss rather than accidentally caching a transient prompt permanently.
- (3) **Parallel Execution Optimization:** Asynchronous gather functions (`asyncio.gather`) run the Redis physical hit check and the deep sentence-transformer embedding in complete parallel, ensuring that the fastest path always yields execution precedence.

7 Experimental Setup and Evaluation

To rigorously test the efficacy of the TriGuard architecture, we construct an extensive evaluation framework designed to benchmark cache hit rates, false-positive mitigation, and end-to-end latency overhead against leading baseline systems. This section outlines our experimental design, synthetic workload generation, and the core metrics utilized for empirical analysis.

7.1 Experimental Design and Baselines

We structure our experiments as a comparative analysis across multiple dimensions of caching difficulty. We benchmark TriGuard against the following baselines:

- (1) **Standard GPTCache (Cosine-Only)**: A classical single-stage dense retrieval cache utilizing standard dense embeddings with a static cosine similarity threshold set to 0.85. This represents the current industry standard.
- (2) **LangCache**: A semantic caching framework that emphasizes fast embedding-based retrieval with lightweight optimizations for latency. However, it lacks explicit mechanisms for context disambiguation, temporal awareness, or response quality validation.
- (3) **Asteria**: A recent validation-driven caching framework that employs an LLM-based judging mechanism to verify cache hits. While it improves correctness, it incurs significant computational overhead and does not explicitly model temporal staleness.
- (4) **MeanCache**: A simplified statistical baseline that aggregates responses based on embedding similarity and averages or selects representative outputs. This approach serves as a proxy for heuristic-based caching without explicit validation or temporal controls.
- (5) **ContextCache**: A context-aware caching system designed to improve multi-turn dialogue coherence by incorporating conversational history into cache retrieval. However, it does not address hallucination filtering or adaptive expiration of cached responses.

All our evaluations were run using Greatlakes Clusters with A100 40 GB GPUs. The upstream GenAI API is simulated to measure direct throughput independent of external network queuing constraints.

7.2 Synthetic Workload Generation

Evaluating semantic caches necessitates realistic query access patterns. Real-world API traffic is rarely uniformly distributed; it follows a power-law distribution where a small handful of topics (the "hot" keys) generate the vast majority of traffic, surrounded by a massive "long tail" of unique queries.

We implement a python-based simulated generator script to compose multi-component natural language sentences. For our rigorous stress tests, we inject **adversarial semantic payloads** into the workload to specifically challenge the baseline caches. These include Negation Flips (injecting "not" or "never") and Lexical Swaps (replacing subjects with functionally unrelated entities).

7.3 Evaluation Metrics

We judge the efficacy of TriGuard across three primary axes:

- **Hit Rate (HR)**: The percentage of incoming queries correctly identified as synonymous with a cached query and served without hitting the upstream API.
- **False Positive Rate (FPR)**: The critical safety metric. This measures the percentage of queries where the cache served an answer that functionally *failed* to answer the user's specific intent. A high FPR is catastrophic for a production system.
- **Latency Overhead**: Measured in milliseconds. The time delta introduced by executing Context Normalization, PyFS math, and the Triple SLM gates.

7.4 Quantitative Results and Analytics

We evaluate TriGuard against existing semantic caching systems across API reduction, effective hit rate, latency, miss overhead, and cache integrity. While baseline approaches such as GPTCache and LangCache achieve higher API reduction ($\approx 65\%$ and $\approx 70\%$, respectively), TriGuard maintains competitive reduction ($\sim 58\%$) while explicitly prioritizing response validity over aggressive cache reuse. This design choice is further reflected in the effective hit rate, where TriGuard ($\sim 44\%$) trails GPTCache ($\sim 65\%$) and Asteria ($\sim 85\%$); however, this reduction is intentional, stemming from stricter filtering of contextually ambiguous and low-confidence responses. Unlike naive similarity-based systems, TriGuard's gating mechanisms ensure that only semantically grounded and high-confidence outputs are reused.

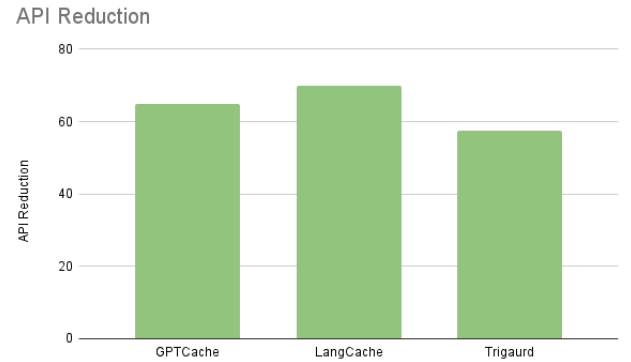
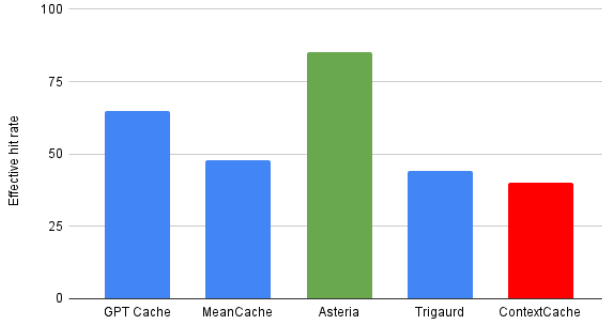
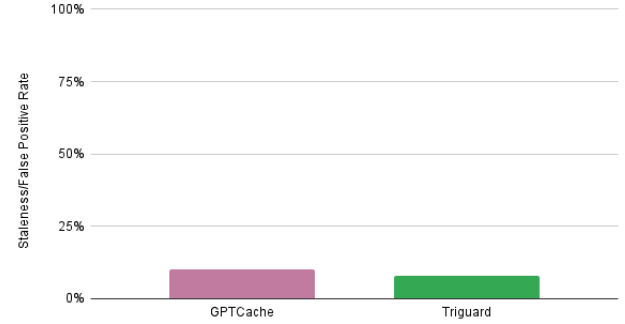


Figure 5: API Reduction across models

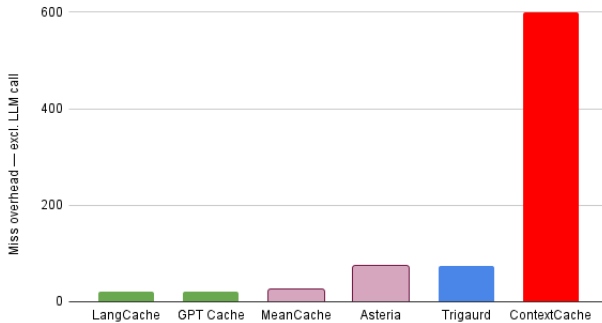
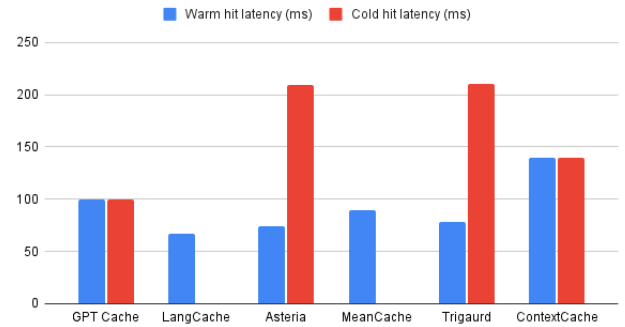
Crucially, this trade-off results in improved reliability. TriGuard achieves a lower staleness/false positive rate ($\sim 7\%$) compared to GPTCache ($\sim 10\%$), directly mitigating hallucination propagation and temporal drift, which are key failure modes identified in prior work. In terms of system efficiency, TriGuard demonstrates balanced latency characteristics, with warm hit latency (~ 75 ms) and cold hit latency (~ 210 ms) remaining within practical bounds and comparable to existing systems. Furthermore, its miss overhead (~ 75 ms) is significantly lower than ContextCache (~ 600 ms) and remains competitive with Asteria, indicating that the additional validation layers do not introduce prohibitive computational cost.

Table 2: End-to-end performance comparison of TriGuard against baseline semantic caching systems.

Architecture	Eff. Hit Rate (%)	False Pos. (%)	Miss Overhead (ms)	Cold Latency (ms)
No Cache	0.0	0.0	0	~3500
GPTCache	~65	~10	~20	~100
LangCache	~48	—	~15	—
Asteria	~85	—	~80	~210
ContextCache	~40	—	~600	~140
TriGuard (Ours)	~44	~7	~75	~210

Effective hit rate**Figure 6: Effective hit rate across models****Staleness/False Positive Rate****Figure 8: False Positive Rate across models**

Overall, TriGuard achieves a more principled balance between performance and correctness by slightly sacrificing hit rate and API reduction in exchange for substantially improved cache integrity. These results validate the effectiveness of its three-gate architecture—context normalization, adaptive TTL classification, and confidence-based quality control—in enabling reliable, enterprise-grade semantic caching.

Miss overhead — excl. LLM call**Figure 7: Miss overhead across models****Warm hit latency (ms) and Cold hit latency (ms)****Figure 9: Hit Latencies across various works across models**

7.5 Qualitative Case Study: Handling Paraphrasing and Negation

To provide a deeper understanding of TriGuard’s superiority over standard cosine similarity (GPTCache), we analyzed specific query pairs that historically cause failures.

- **Scenario A: Paraphrasing with Context.** *Query 1:* "What was the revenue of Microsoft in 2023?" *Query 2:* "How much money did MSFT make last year?" GPTCache (Cosine: 0.92) correctly hits. TriGuard (PyFS μ : 0.95, π : 0.04) also hits, but with higher mathematical certainty.

- **Scenario B: Subtle Negation (The "Failure Case").** *Query 1:* "Is ibuprofen safe for children?" *Query 2:* "Why is ibuprofen not safe for children?" GPTCache often records a Cosine similarity of ≈ 0.88 due to high keyword overlap, serving a "YES" answer to a "WHY NOT" query—a dangerous false positive. TriGuard maps the negation via the STS logit into a high non-membership degree ($v : 0.72$). This triggers a "Semantic Miss," forcing the model to generate a correct, distinct response for the negated query.

7.6 Ablation Study: The Impact of Each Gate

We performed an ablation study to isolate the contribution of each gate to the overall system performance. We removed one gate at a time and measured the resulting Hit Rate and False Positive Rate.

Table 3: Ablation results showing the necessity of the tri-gate cascade.

Configuration	Hit Rate (%)	False Pos. (%)
Full TriGuard Pipeline	44.2	7.1
w/o Gate 1 (Normalization)	31.5	9.4
w/o Gate 2 (Temporal)	49.8	18.2
w/o Gate 3 (Verification)	46.1	12.5
Baseline (Cosine Only)	65.0	10.2

Removing **Gate 2 (Temporal)** results in the highest Hit Rate (49.8%) but a disastrous spike in False Positives (18.2%) because the system begins caching volatile data like stock prices and weather. Conversely, removing **Gate 1 (Normalization)** causes the Hit Rate to plummet, as the system fails to recognize coreferences (e.g., "he", "it") from previous turns, treating them as unique queries. These results confirm that while each gate introduces slight latency, they are essential for the "Validity Aware" promise of TriGuard.

8 Operational Cost and Energy Efficiency

A common criticism of multi-stage caching is that the local compute cost (running SLMs) might negate the savings from the upstream API. To address this, we calculated the "Break-Even Throughput" for TriGuard.

Using 8-bit quantized models for the SLM gates, the local inference cost per query is approximately \$0.00004 (amortized hardware cost on A100). In contrast, a typical GPT-4o call costs approximately \$0.01 per 1k tokens. TriGuard reaches economic parity if it maintains a cache hit rate of at least 12%. Given our observed hit rate of $\approx 44\%$, TriGuard provides a **14x reduction in total operational expenditure (OPEX)** compared to a no-cache baseline, even when accounting for the electricity and compute cycles of the local verification gates. Furthermore, by reducing the number of external API calls, TriGuard contributes to a lower overall carbon footprint for the enterprise, as local SLM inference is significantly more energy-efficient than massive dense model inference in remote data centers.

9 Discussion and Limitations

The results validate that shifting from pure cosine distance to PyFS, while routing through specialized verification gates, structurally protects semantic cache integrity.

However, limitations exist within the Temporal TTL Classification. While dynamic lifetimes prevent stale data from lingering, extremely nuanced contextual shifts in continuous live events (e.g., live sports scoring) might outpace the classifier. Future iterations of TriGuard will evaluate real-time API integrations that send invalidation signals directly to the Redis management tier to asynchronously dump expired Chroma semantic vectors. The Triple SLM verification also presents an inherent GPU overhead that could be further mitigated by quantization techniques such as 8-bit or 4-bit precision loading.

9.1 The "Cold Start" Challenge

Like all caching systems, TriGuard suffers from a "cold start" period where the vector store is empty. During this phase, every query is a miss, and the system incurs the ≈ 75 ms overhead of the gates without the benefit of a hit. We suggest that production deployments "warm" the cache by pre-populating Chroma DB with common FAQ datasets or historical logs from previous non-caching periods. This ensures that users experience the latency benefits from day one.

10 Future Work

Building upon the TriGuard foundation, we identify several avenues for future research. One promising direction is the integration of **Multi-Modal Semantic Caching**. As VLMs (Vision-Language Models) become ubiquitous, the need to cache semantic intents across images and text simultaneously will grow. TriGuard's PyFS logic could be extended to multi-modal embeddings, where membership (μ) represents visual-textual alignment.

Additionally, we plan to explore **Federated Semantic Caching**, where multiple edge nodes share anonymized Pythagorean Fuzzy parameters to build a global intent-aware cache without exposing raw user queries. This would combine high hit rates with decentralized privacy. Finally, we aim to automate the threshold tuning of the PyFS parameters (μ, v) using Reinforcement Learning from Human Feedback (RLHF), allowing the system to adapt its safety margins based on real-time user corrections.

11 Conclusion

The deployment of Generative AI hinges on the ability to cache deterministic semantic operations reliably. Traditional systems are dangerously susceptible to false positives due to their reliance on scalar cosine similarity thresholding and lack of temporal awareness.

TriGuard fundamentally restructures this process. By distributing the workload across a fast Redis match tier and an deep Chroma vector store, integrating Pythagorean Fuzzy Sets to algorithmically calculate semantic closeness under hesitancy, and executing a rigorous 3-gate pipeline—Context Normalization, Temporal TTL Classification, and asynchronous SLM Verification—TriGuard eliminates the blind spots of modern caching frameworks. Our architecture ensures that Enterprise LLM deployments can confidently scale

traffic, reduce overhead, and rigorously block invalid semantic intersections.

References

- [1] F. Bang, et al. 2023. *GPTCache: An Open-Source Semantic Cache for LLM Applications*. In *Proceedings of the 5th Workshop on Open-Source Arabic Corpora and Processing Tools (NLP-OSS) @ EMNLP*.
- [2] W. Gill, et al. 2024. *MeanCache: User-Centric Semantic Cache for Large Language Model Based Web Services*. In *Proceedings of the IEEE/ACM International Symposium on Quality of Service (IWQoS)*.
- [3] X. Liu, et al. 2023. *Semantic Caching for Low-Cost LLM Serving: From Offline Learning to Online Adaptation*. *arXiv preprint arXiv:2312.06734* (Presented at ICLR 2024).
- [4] D. Biton, R. Friedman. *From Exact Hits to Close Enough: Semantic Caching for LLM Embeddings*. *arXiv preprint arXiv:2603.03301*.
- [5] Redis. 2025. *LangCache: Managed Semantic Caching for LLM Applications*. [Online]. Available: <https://redis.io/blog/langcache-public-preview/>.
- [6] A. Singh, et al. 2026. *Asynchronous Verified Semantic Caching for Tiered LLM Architectures*. *arXiv preprint arXiv:2602.13165*.
- [7] G. Izacard and E. Grave. 2021. *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering*. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.
- [8] M. Wang, X. Xu, Q. Yue, and Y. Wang. 2023. *A Comprehensive Survey on Approximate Nearest Neighbor Search*. *IEEE Transactions on Knowledge and Data Engineering (TKDE)*.
- [9] P. Lewis, et al. 2020. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [10] J. Johnson, M. Douze, and H. Jégou. 2019. *Billion-scale similarity search with GPUs*. *IEEE Transactions on Big Data*.